MLSMM2156 - Recommender Systems - Group 3

Design and Development of a Movie Recommendation System

Supervisor:

Corentin Vande Kerckhove

Students:

Lenny Andry

Mohamed El Mohcine

Arthur Ottevaere

Submission date: June 6, 2026

Contents

1. Introduction **1**

2. Live demo **2**

3. Experimental setup **2**

3.1. Dataset & Exploratory Analysis 2

3.2. Evaluation Protocols 3

3.3. Evaluation Metrics 4

3.3.1. Error-Based Metrics 4

3.3.2. Ranking and Retrieval Metrics 4

3.3.3. Catalog and Diversity Metrics 5

3.4. Recommendation Models 6

3.4.1. User-based Collaborative Filtering 6

3.4.2. Content-Based Filtering 7

3.4.3. Latent Factor Models 8

4. Results & Model Comparison **10**

4.1. Rating prediction (75/25 split) 10

4.2. Top-N ranking — full-catalogue Leave-One-Out 12

4.3. Top-N ranking — sampled protocol (1 positive + 99 negatives) 12

4.4. Catalogue diversity (full mode, top-40) 13

4.5. Synthesis 14

5. Discussion & Conclusion **15**

5.1. Summary 15

5.2. Strengths 15

5.3. Limitations 15

Appendix **18**

Appendix 1 - Ratings distribution 18

Appendix 2 - Sparsity Visualization 18

Appendix 3 - Movie Release Year Distribution 19

Appendix 4 - Rating Distribution by Genre 19

Declaration on the Responsible Use of Generative AI

In accordance with the guidelines of the Louvain School of Management (LSM) and UCLouvain on the responsible use of generative artificial intelligence, we disclose how generative AI tools were used in the preparation of this work, and we confirm that our use complied with the principles of *Responsibility*, *Transparency*, *Authenticity* and *Critical Evaluation*.

During this project, we made use of generative AI tools, in particular Anthropic's *Claude* large language models, accessed mainly through *Claude Code*, an agentic command-line assistant for software engineering. These tools were used in an assistive capacity for tasks such as:

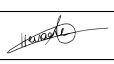
- code generation, refactoring and debugging of the recommender system;
- improving the wording, spelling and grammar of the written report;
- exploring design alternatives and helping to structure our ideas;
- searching for and explaining technical concepts.

The use of these tools was strictly assistive. The conceptual decisions, the overall design, the choice and justification of the methodology, the validation of the results and the final integration of all components were carried out by us, under our direct supervision. Every AI-generated output was critically reviewed, verified, tested and, where necessary, corrected and adapted. We did not provide the AI tools with any confidential, sensitive or proprietary data. We take full responsibility for the content, correctness and integrity of this work, including any portion that was initially produced or assisted by AI.

Statements on the responsible use of generative AI

Statement	**Answer**
The content of our work is our original input, even if we used generative AI to help prepare it.	Yes
We master the theories, tools and methods that we use for this work.	Yes
We verified that the output presented is accurate and valid, and that it contains no hallucination, no false reference, etc.	Yes
We acknowledge that we master the final output and that we are able to explain it and answer questions about it.	Yes
We made sure not to provide generative AI tools with access to confidential data or information.	Yes

Signature

Author last name, first name(s)	**Date**	**Signature**
Andry Lenny	June 4th 2026	
El Mohcine Mohamed Amine	June 4th 2026	
Ottevaere Arthur	June 4th 2026	

1. Introduction

In the contemporary digital ecosystem, media streaming platforms have fundamentally shifted the paradigm of content consumption. While users now have access to massive entertainment libraries, they simultaneously face information overload, where the sheer volume of choices can paralyze decision-making. Consequently, robust personalized filtering mechanisms are an infrastructural necessity to navigate this high-dimensional space and guide users toward relevant items. The primary objective of this project is to build a **full-featured movie recommender system** that seamlessly integrates advanced predictive modeling with a functional user interface.

The dataset comprises **381,181 historical interactions** from **1,000 users** across **8,737 movies**. User preferences are encoded via explicit ratings ranging from 0.5 to 5.0, resulting in a **matrix sparsity of 95.64%**. To mitigate this sparsity and enhance recommendations, the core matrix is enriched with auxiliary metadata from **The Movie Database (TMDB)** and high-dimensional **MovieLens genome scores** spanning 1,128 semantic dimensions.

The architecture integrates **four core recommendation models** to balance prediction accuracy, ranking performance, and catalog diversity: a **User-Based Collaborative Filtering** approach comparing a Pearson Baseline with shrinkage to a custom Jaccard similarity, a **Content-Based Filtering** pipeline using **Ridge CV**, an **Implicit Alternating Least Squares (iALS)** engine, and a **Bayesian Personalized Ranking (BPR)** framework with its **BPR+Novelty** variant. The system is deployed as a web application with a **FastAPI backend** and **JavaScript frontend**, featuring interactive carousels, detailed movie pages, watchlists, and user profiles.

Moving beyond interface features, the primary analytical focus of this report is to rigorously benchmark the developed recommendation architectures. The system's operational performance is systematically evaluated across three fundamental dimensions. First, **rating prediction error** is quantified through Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) to establish baseline statistical fidelity. Second, **top-N ranking efficiency** and retrieval performance are evaluated under full-catalog and negative sampling constraints, measured via Hit Rate (HR@K) and Normalized Discounted Cumulative Gain (NDCG@K). Finally, **catalog diversity** and **distributional fairness** are assessed via Intra-List Diversity (ILD), catalog coverage, and Mean Inverse User Frequency (MIUF) to guarantee balanced recommendations that resist standard popularity bias.

The remainder of this report is organized as follows. Section 2 presents the URL link to the live video demonstration, highlighting the web application's core workflow. Section 3 details the experimental setup, including exploratory data analysis, evaluation methodology, model specifications, and hyperparameter grids. Section 4 provides a consolidated comparative analysis of the experimental results. Section 5 discusses the system's structural strengths and inherent limitations, and concludes with a synthesis of the overall approach.

2. Live demo

To demonstrate the operational capabilities and interactive features of the developed movie recommender system, a video demonstration has been recorded. The walkthrough showcases the end-to-end web application pipeline, including user onboarding, personalized carousels, and the integrated Gemini chatbot. The video is available at:

`https://www.youtube.com/watch?v=rvToIaW10m4`

3. Experimental setup

3.1. Dataset & Exploratory Analysis

The empirical validation of the implemented architectures relies on the official hackathon dataset, which aggregates 381,181 explicit interactions (ratings from 0.5 to 5.0 with increments of 0.5) from 1,000 unique users across 8,737 distinct movies. Exploratory analysis reveals a pronounced positivity bias, with a global mean rating of 3.45 and heavy score concentration at the 4.0 and 5.0 thresholds (see Appendix 1). A primary structural challenge resides in the extreme matrix sparsity, quantified at 95.64%, meaning that only 4.36% of the potential interaction cells contain an explicit preference score (see Appendix 2).

The catalog also exhibits a strong contemporary skew, with a median release year of 1997 and a dominant volume of 2,440 films produced during the 2000s (see Appendix 3). Beyond temporal trends, a major structural imbalance exists across genres (see Appendix 4). Mainstream genres dominate interaction volumes, led by Drama with over 160,000 ratings, followed by Comedy and Action. Conversely, niche genres like Western, Documentary, and Film-Noir suffer from historical data scarcity, with Film-Noir receiving fewer than 5,000 ratings. However, low volume does not imply low appreciation; Film-Noir, War, and Documentary boast the highest average ratings at 3.89, 3.72, and 3.71 respectively, while popular genres like Horror underperform at a 3.22 average.

.

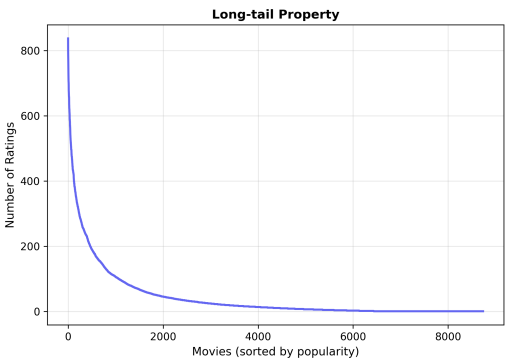


Figure 1: Interaction distribution per movie showing the long-tail behavior.

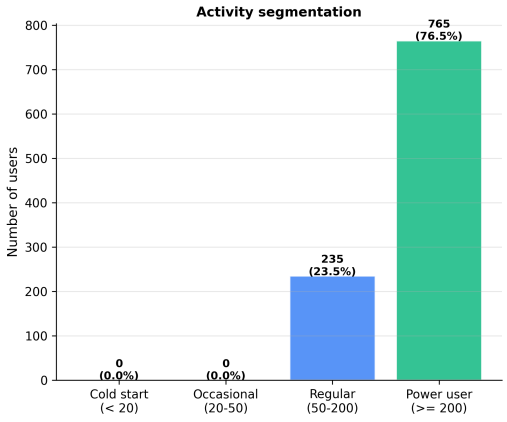


Figure 2: Historical ratings per user

Analysis of the operational interaction distributions highlights a contrast between the two dimensions of the matrix. The distribution of ratings per movie reveals an acute popularity bias following a severe power-law behavior (Abdollahpouri et al., 2019), where a small core of blockbuster films gathers a disproportionate fraction of the total rating volume, as illustrated in Figure 1. Conversely, the distribution of ratings per user, detailed in Figure 2, reveals an exceptionally active user base with an heterogeneity skewed toward high-volume profiling. Specifically, the minimum activity threshold stands around 50 ratings per profile, while 76.5% of the 1,000 users have evaluated more than 200 movies each, stretching up to nearly 2,000 ratings for the most hyperactive power-users. This high volume of historical data per user provides a solid training foundation for collaborative models. However, the global matrix remains highly sparse at 95.64%. This justifies the use of TMDB metadata and shrinkage constraints to improve prediction stability, especially for users with fewer ratings.

3.2. Evaluation Protocols

To perform a mathematically rigorous and multidimensional benchmark of the implemented recommender systems, the evaluation pipeline is segmented into three complementary experimental protocols. Each protocol replicates a specific operational setting and targets a distinct performance dimension of the recommendation process.

First, a **random split-based protocol** is implemented via the `generate_split_predictions` pipeline. It partitions the explicit interaction matrix using a fixed 75% training size and a 25% test size configuration, enforcing a deterministic seeding (`random_state=42`) to guarantee strict replicability. This protocol isolates the models' capacity to minimize rating prediction errors on a holdout sample of known historical preferences. However, because minimizing rating errors does not fundamentally reflect the user experience of interacting with a ranked recommendation feed, this protocol is exclusively restricted to error-based analytics.

Second, a **Leave-One-Out (LOO)** protocol is deployed to assess ranking and retrieval performance under two distinct operational paradigms:

- **Full-Catalog Ranking:** Executed via `generate_loo_top_n`, this protocol uses a chronological leave-one-item-out strategy (`random_state=1`) where the last interaction of each user profile is withheld as the positive test target, while the remaining sequence builds the training set. Recommendations are generated by scoring and sorting the complete `anti_testset`, which combines this hidden target movie with all unspecified items in the catalog to extract the optimal top-N items.
- **Negative Sampling [ns]:** To mitigate computational constraints, a parallel protocol is established via `generate_loo_neg_sampling_top_n`, adhering strictly to the literature standards formalized by He et al. (2017). For each hidden positive item in the test set, the algorithm samples 99 random unrated items that act as negative distractors, yielding a fixed evaluation pool of 100 candidate items per user. Although computationally efficient, this

sampled protocol introduces structural evaluation biases depending on the item popularity distribution, as demonstrated by Krichene and Rendle (2020). Running and reporting both full-catalog and negative-sampled metrics in parallel provides a transparent, unbiased overview of top-N ranking capabilities.

Third, a **full-mode protocol** is deployed via `generate_full_top_n` to evaluate the intrinsic structural properties of the recommendation lists across the entire catalog. Unlike the previous configurations, the models are trained on 100% of the available historical interactions. The algorithm then generates a fixed top-N recommendation list for every user profile from the complete set of unrated movies. This protocol serves as the foundation for computing macro-metrics that assess system coverage, semantic diversity, and distributional fairness across the catalog. It also serves as the operational hook for a popularity-based re-ranking pipeline implemented via `get_top_n_reranked`. This function computes a combined score defined as $S = \hat{r}_{ui} + \alpha \cdot \log(1 + \text{freq}_i)$, where the hyperparameter α calibrates the exact trade-off between the model's predicted rating and the item's historical popularity.

3.3. Evaluation Metrics

To map the operational capabilities of the systems across the established protocols, a comprehensive suite of metrics is deployed, capturing error magnitude, ranking performance, catalog coverage, distributional fairness, and semantic diversity.

3.3.1. Error-Based Metrics

Evaluated exclusively under the 75/25 random split protocol, rating prediction fidelity is quantified using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE):

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} (\hat{r}_{ui} - r_{ui})^2} \quad , \quad \text{MAE} = \frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} |\hat{r}_{ui} - r_{ui}| \tag{1}$$

where $\mathcal{T}$ represents the test set of explicit interactions, r_{ui} denotes the true historical rating, and $\hat{r}_{ui}$ is the score predicted by the model. While MAE treats all operational errors uniformly, RMSE penalizes larger prediction deviations more severely, serving as a critical diagnostic tool for baseline rating stability.

3.3.2. Ranking and Retrieval Metrics

To measure the models' capacity to successfully rank the hidden target item within the top-K recommendations under both full-catalog and negative-sampled Leave-One-Out protocols, Hit Rate (HR@K) and Normalized Discounted Cumulative Gain (NDCG@K) are computed at evaluation thresholds $K \in \{5, 10, 20\}$.

The Hit Rate acts as a binary retrieval success indicator, defined as:

$$\text{HR@}K = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}(\text{rank}_u(i^*) \leq K) \tag{2}$$

where $\mathcal{U}$ is the evaluation user base, i^* represents the hidden positive item, $\text{rank}_u(i^*)$ is its positional index in the generated list, and $\mathbb{I}(\cdot)$ is the indicator function.

To account for the exact positioning of the item within the top-K slots, NDCG@K introduces a logarithmic position decay:

$$\text{NDCG@}K = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\log(2)}{\log(\text{rank}_u(i^*) + 1)} \cdot \mathbb{I}(\text{rank}_u(i^*) \leq K) \tag{3}$$

Evaluating these metrics at $K = 10$ provides the core baseline for ranking efficiency, balancing consumer visibility constraints with algorithm retrieval depth (Cremonesi et al., 2010).

3.3.3. Catalog and Diversity Metrics

Computed in full-mode using a fixed list length of $N = 40$ items, macro-metrics assess the systemic structural behavior of the recommendation engines:

- **Coverage:** Quantifies the percentage of unique items in the 8,737 catalog that appear at least once across all users' top-40 recommendation lists, exposing any algorithmic tendency toward severe catalog constriction.
- **Intra-List Diversity (ILD):** Measures the average semantic distance between all pairs of items recommended within a single user's list, utilizing normalized genre vectors extracted from TMDB metadata, corresponding to the standard course formulation (Castells 2015):

$$ILD = \frac{1}{|R|(|R| - 1)} \sum_{i \in R} \sum_{j \in R} d(i, j) \tag{4}$$

  where R is the recommendation list, and $d(i, j) = 1 - \cos(\mathbf{g}_i, \mathbf{g}_j)$ represents the distance function computed as the complement of the cosine similarity between the genre vectors of movies i and j.

- **Mean Inverse User Frequency (MIUF):** Evaluates the structural novelty and non-popularity bias of the generated recommendations. By utilizing the precomputed item frequency distribution (`item_freq`), it quantifies whether the system frequently explores the long-tail catalog or exhibits a restrictive popularity bias by heavily favoring mainstream, high-volume items, according to the standard course formulation (Castells 2015) :

$$MIUF = \frac{1}{|R|} \sum_{i \in R} -\log_2 \frac{|\mathcal{U}_i|}{|\mathcal{U}|} \tag{5}$$

  where $|R|$ is the size of the recommendation list, $|\mathcal{U}_i|$ is the number of unique users who interacted with item i, and $|\mathcal{U}|$ represents the total number of users in the dataset.

3.4. Recommendation Models

This section presents the six models retained in `configs.py`. Four are deployed in the application via the carousels *"Recommended for You"*, *"Viewers Like You Also Watched"*, *"Top Picks For You"* and *"Discover Something New"*. The two remaining serve as reference baselines: classic BPR and `UserBased_Tuned_Jaccard`, the latter introduced to meet the requirement of implementing at least one similarity metric absent from the Surprise library.

3.4.1. User-based Collaborative Filtering

> *Neighbourhood-based collaborative filtering rests on the hypothesis that users with similar preferences will continue to agree (Resnick et al., 1994). For a user u and an item i, the k most similar neighbours who rated i are identified, and their ratings are combined as a weighted average centred on individual means:*

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_k(u,i)} \mathrm{sim}(u,v)\,(r_{vi} - \bar{r}_v)}{\sum_{v \in N_k(u,i)} |\mathrm{sim}(u,v)|} \tag{6}$$

If fewer than `min_k` neighbours are available, the prediction falls back to $\bar{r}_u$.

UserBased_Pearson_Natif (carousel *"Viewers Like You Also Watched"*). This variant uses `KNNBaseline` from Surprise, whose prediction integrates baseline effects $b_{ui} = \mu + b_u + b_i$ to centre the contributions of neighbours (Koren et al., 2009). The Pearson Baseline similarity with shrinkage attenuates the Pearson coefficient centred on baselines proportionally to the available support ($\beta = 100$ in Surprise), bringing it towards zero when $|I_{uv}|$ is small (Bell and Koren, 2007).

$$\mathrm{sim}_{\mathrm{PB}}(u,v) = \frac{|I_{uv}| - 1}{|I_{uv}| - 1 + \beta} \cdot \hat{\rho}_{uv} \tag{7}$$

Hyperparameters. Explored via RMSE curves on a 75/25 split (`user_based.ipynb`), with $k \in \{5, \ldots, 100\}$, `min_k` $\in \{1, \ldots, 10\}$ and `min_support` $\in \{1, \ldots, 10\}$. The RMSE stabilises at $k = 40$, offering the best accuracy/diversity balance. The value `min_k` $= 2$ was set manually: `min_k` $= 1$ produces predictions that are too unreliable, while higher values increase fallbacks to $\bar{r}_u$. The threshold `min_support` $= 3$ filters spurious similarities without degrading coverage. Configuration retained: $k = 40$, `min_k` $= 2$, `min_support` $= 3$, $\alpha_{rr} = 0.5$.

UserBased_Tuned_Jaccard (comparison baseline). This variant uses the class `UserBased_tuned` from `models.py`, which computes the similarity matrix entirely from scratch. The similarity metric used is Jaccard. The Jaccard similarity compares only the sets of rated items, without taking into account the rating values (Desrosiers and Karypis, 2011):

$$\text{sim}_{\text{Jac}}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|} \tag{8}$$

Hyperparameters. Selected via `GridSearchCV` with 5 folds (`user_based_tuning.ipynb`), grid: $k \in \{20, 30, 40, 50\}$, `min_k` $\in \{3, 5, 7\}$, `min_support` $\in \{3, 5\}$, Jaccard similarity. The combination $k = 40$, `min_k` $= 3$, Jaccard similarity offered the best RMSE/HR@K/coverage balance. The value `min_k` $= 2$ was set manually: higher values increase the fallback rate to $\bar{r}_u$, while `min_k` $= 2$ offers a better accuracy/coverage balance and aligns with the configuration of `UserBased_Pearson_Natif`. Final configuration: $k = 40$, `min_k` $= 2$, `min_support` $= 3$, similarity = Jaccard, $\alpha_{rr} = 0.5$.

Strengths and limitations. User-based filtering is easy to interpret, but becomes inefficient on large user bases (Linden et al., 2003) and struggles to recommend content to new users.

3.4.2. Content-Based Filtering

Content-based filtering relies on item attributes to build a preference profile specific to each user (Pazzani and Billsus, 2007), without requiring data from other users, making it less sensitive to the user cold-start problem (Lops et al., 2011). For each user u, a regressor is trained on the set of rated items ($\mathcal{I}_u$), with features $\mathbf{x}_i \in \mathbb{R}^d$ as explanatory variables and the rating r_{ui} as the target variable. The feature vector ($\mathbf{x}_i$) groups the descriptive information of films: genome scores (1 128 dimensions, standardised), TF-IDF on user tags (2 000 features, unigrams and bigrams, attenuated weighting of high frequencies, ignored for tags appearing in fewer than 3 films), genres (TF-IDF), normalised release year and decade indicators, and TMDB metadata (normalised film duration, language, country, studios, directors, cast, keywords, normalised budget, normalised release date, TF-IDF synopsis). The resulting space exceeds 4 000 dimensions, which motivates regularisation to limit overfitting.

ContentBased_ridge_cv (carousel *"Recommended for You"*). For each user u, a Ridge model is trained independently, minimising the regularised squared error (Hoerl and Kennard, 1970):

$$\min_{\mathbf{w}_u} \sum_{i \in \mathcal{I}_u} (r_{ui} - \mathbf{w}_u^\top \mathbf{x}_i)^2 + \alpha \|\mathbf{w}_u\|_2^2 \quad \Rightarrow \quad \hat{\mathbf{w}}_u = (\mathbf{X}_u^\top \mathbf{X}_u + \alpha \mathbf{I})^{-1} \mathbf{X}_u^\top \mathbf{y}_u \tag{9}$$

where $\mathbf{w}_u \in \mathbb{R}^d$ is the weight vector specific to user u, $\mathcal{I}_u$ is the set of items rated by u, and α is the L2 regularisation coefficient. The analytical solution is $\hat{\mathbf{w}}_u = (\mathbf{X}_u^\top \mathbf{X}_u + \alpha \mathbf{I})^{-1} \mathbf{X}_u^\top \mathbf{y}_u$.

Hyperparameters. The parameter α is selected automatically via `RidgeCV` on a grid from 10^{-4} to 10^5. Ridge offers good performance (RMSE) on this dense feature space and native interpretability via $\mathbf{w}_u$, which captures the relative importance of each feature per user. The 2 000 TF-IDF features configuration was retained after comparison with 500 and 1 128 features, as it consistently improves RMSE and HR@K without notable overhead. Final configuration: `ridge_cv`, `all_content_tmdb_tags2000`.

Strengths and limitations. It is the only model capable of recommending items without ratings (catalogue cold-start). Its main limitation is the risk of filter bubbles (Jannach et al., 2015).

3.4.3. Latent Factor Models

Latent factor models rely on a matrix decomposition of the user-item interaction matrix (Koren et al., 2009). Each user u and each item i are represented by dense vectors $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^f$, and the prediction is the dot product $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$. Three variants have been implemented and evaluated in this project.

ModeliALS (carousel *"Top Picks For You"*). The model `ModeliALS` implements the iALS algorithm (implicit Alternating Least Squares), also known as WRMF (Weighted Regularized Matrix Factorization) (Hu et al., 2008). Unlike classical matrix factorization models that optimise only observed entries, iALS processes the full matrix by distinguishing preference and confidence. In this project, explicit ratings are used as a confidence proxy: the higher a user's rating, the stronger the confidence in their preference. The confidence weight is defined as:

$$c_{ui} = 1 + \alpha \cdot r_{ui} \tag{10}$$

With $\alpha = 40$ and ratings $r_{ui} \in [0.5; 5.0]$, a 5-star film obtains a confidence of $c_{ui} = 201$ against $c_{ui} = 21$ for a 0.5-star film, in line with the original formulation of (Hu et al., 2008).

The function to minimise is a weighted squared error over the entire user-item matrix with L2 regularisation:

$$\mathcal{L}_{iALS} = \sum_{u,i} c_{ui} \left(p_{ui} - \mathbf{p}_u^\top \mathbf{q}_i\right)^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2\right) \tag{11}$$

where $p_{ui} = 1$ if user u has rated item i, and $p_{ui} = 0$ otherwise (binary preference signal). The unobserved terms ($c_{ui} = 1$ for unrated items) nevertheless contribute to the optimisation, which fundamentally distinguishes iALS from classical MF approaches (Pan et al., 2008). Minimisation is performed by Alternating Least Squares: user factors are updated by fixing item factors, and vice versa, each sub-problem admitting a closed-form analytical solution.

Hyperparameters. The retained configuration — $f = 50$ factors, $T = 20$ iterations, $\lambda = 0.01$ and $\alpha = 40$ — corresponds to the values recommended by (Hu et al., 2008) and validated on this dataset: $f = 50$ provides sufficient representational capacity without overfitting the model on the hackathon dataset, $\alpha = 40$ is the default choice for implicit feedback, $\lambda = 0.01$ prevents overfitting, and 20 ALS iterations are sufficient for convergence on this data volume.

ModelBPR (comparison baseline). The model `ModelBPR` (Bayesian Personalized Ranking) implements the BPR-MF algorithm (Rendle et al., 2009), present in the evaluation as a comparison point to quantify the gain brought by the popularity penalty of BPR_Novelty. Unlike the previous models, BPR does not optimise a rating prediction error, but a pairwise ranking loss. For any user u, if i is an item they have rated and j an item they have not rated, then u prefers i to j: $u \succ_i j \Leftrightarrow \hat{r}_{ui} > \hat{r}_{uj}$. The training signal is therefore binary: having rated an item ($= 1$) versus not having rated it ($= 0$), regardless of the rating value.

The optimisation maximises the posterior log-likelihood over the set of triplets (u, i, j):

$$\mathcal{L}_{BPR} = \sum_{(u,i,j) \in \mathcal{D}_S} \log \sigma(\hat{r}_{ui} - \hat{r}_{uj}) - \lambda \|\Theta\|^2 \tag{12}$$

where $\mathcal{D}_S = \{(u, i, j) \mid i \in \mathcal{I}_u^+,\ j \notin \mathcal{I}_u^+\}$ is the set of training triplets, $\sigma(\cdot)$ is the sigmoid function, $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$ is the score of the positive item, $\hat{r}_{uj} = \mathbf{p}_u^\top \mathbf{q}_j$ is the score of the negative item, and $\Theta = \{P, Q\}$ is the set of parameters (Rendle et al., 2009). Optimisation is performed by stochastic gradient descent (SGD) on small batches of randomly sampled triplets.

> ***Hyperparameters.*** The configuration $f = 64$ factors, $\eta = 0.01$ (learning rate), $\lambda = 0.01$ (regularisation) and $T = 100$ iterations is directly inspired by the parameters of (He et al., 2017) on MovieLens-1M. $f = 64$ is higher than for iALS as the pairwise loss is more demanding; 100 SGD iterations are required against 20 for ALS. Final configuration: $f = 64$, $\eta = 0.01$, $\lambda = 0.01$, $T = 100$.

ModelBPRNovelty (carousel *"Discover Something New"*). The model `ModelBPRNovelty` extends BPR-MF by introducing a popularity penalty during ranking, in order to favour relevant but less well-known items (Abdollahpouri et al., 2019). It directly inherits from `ModelBPR` and shares the same training procedure. The difference occurs exclusively during the scoring phase. After BPR training, the raw scores $\hat{r}_{ui}^{BPR}$ are normalised per user, then linearly combined with a logarithmic popularity penalty:

$$\text{score\_adj}(u, i) = (1 - \beta) \cdot \tilde{r}_{ui}^{BPR} - \beta \cdot \frac{\log(1 + \text{pop}(i))}{\log(1 + \text{pop}_{\max})} \tag{13}$$

where $\tilde{r}_{ui}^{BPR} = \dfrac{\hat{r}_{ui}^{BPR} - \min_j \hat{r}_{uj}^{BPR}}{\max_j \hat{r}_{uj}^{BPR} - \min_j \hat{r}_{uj}^{BPR}}$ is the BPR score normalised in $[0, 1]$, $\text{pop}(i) = |\{u : r_{ui} \text{ is observed}\}|$ is the number of users who rated item i in the trainset, and β is the novelty weight. The use of the logarithm compresses the popularity distribution, thus preventing blockbusters from excessively dominating the penalty (Abdollahpouri et al., 2019).

Hyperparameters. The parameter $\beta = 0.2$ constitutes a moderate compromise between relevance and novelty: a value of zero reverts to classic BPR, while a high value ($\beta \geq 0.5$) penalises popular items too strongly, significantly degrading the Hit Rate. The retained value improves diversity metrics (ILD, Coverage) while preserving acceptable ranking accuracy, as confirmed by the direct comparison in the results tables. The other hyperparameters are identical to those of `ModelBPR` in order to isolate the effect of the popularity penalty.

Common strengths and limitations. The three models follow an offline architecture: trained in advance, they produce recommendations in real time via a simple dot product. Their main limitation is the cold-start problem for new items, which do not yet have a learned representation.

4. Results & Model Comparison

No single model dominates every axis, and that is the central finding rather than a limitation. The benchmark was deliberately built on three complementary protocols (Section 3.2) precisely because rating fidelity, ranking quality, and catalogue behaviour reward different mechanisms. The tables below are therefore read not as a leaderboard but as a profile: each model occupies a distinct region of the accuracy–ranking–diversity space, and the deployment in Section 3.4 distributes those strengths across the four carousels.

Before analyzing the results, it is primordial to clarify one specificity of our user-based models. Our recommendation pipeline employs a two-stage architecture to address the inherent "long-tail" bias of neighborhood-based collaborative filtering, which often surfaces highly accurate but overly niche items (Cremonesi et al., 2010). To prevent user disengagement in our carousel interface, where severe position bias dictates that only the very first items capture immediate attention, we apply a subsequent re-ranking step. This re-ranking module explicitly considers the mutual influence between items within the generated list (Pei et al., 2019) by balancing the base algorithm's relevance scores with a popularity factor. Consequently, this approach seamlessly combines deeply personalized niche content with globally appealing "short-head" items, placing familiar visual anchors at the foremost positions of the carousel to build immediate user trust while preserving niche discoveries for users who scroll further.

4.1. Rating prediction (75/25 split)

Model	**RMSE ↓**	**MAE ↓**
Content-Based (Ridge CV)	**0.744**	**0.561**
User-Based (Pearson Baseline)	0.805	0.611
User-Based (Tuned, Jaccard)	0.835	0.636
iALS*	2.768	2.577
BPR*	2.328	2.057
BPR+Novelty	3.489	3.334

Table 1: Rating-prediction error on the 75/25 random split.

> **In accordance with the work of Hu et al. on iALS and Rendle et al. on BPR, these models based on implicit feedback are not trained to reconstruct explicit ratings on the [0.5, 5.0] scale. Their outputs represent uncalibrated preference scores. Therefore, these values are reported for the sake of completeness and are not directly comparable to those of explicit rating models.*

Content-Based Ridge CV records the lowest error of the entire benchmark (Table 1, RMSE = 0.744), narrowly ahead of the Pearson Baseline neighbourhood model (0.805). The result is coherent with the model's construction. Indeed, a per-user ridge regression over a dense, heavily regularised feature space estimates a value directly on the rating scale, and the L2 penalty keeps that estimate stable.

The two neighbourhood variants also separate cleanly, and the direction of the gap is instructive. The Pearson Baseline similarity (RMSE = 0.805) outperforms the custom Jaccard similarity (0.835) on every error metric, a difference that follows directly from the information each one exploits. Jaccard compares only the *sets* of co-rated items and is blind to the rating values themselves (Desrosiers and Karypis, 2011). On an explicit-feedback dataset with a pronounced positivity bias, discarding rating magnitude discards exactly the signal RMSE rewards. The Pearson Baseline instead centres neighbour contributions on the baseline terms $\mu + b_u + b_i$ (Koren et al., 2009) and shrinks low-support correlations toward zero (Bell and Koren, 2007), stabilising the estimate where co-rating evidence is thin. This is the result that justifies deploying the Pearson Baseline variant in the application while retaining the custom Jaccard model as the non-Surprise similarity benchmark required by the guidelines.

The three remaining models sit an order of magnitude higher, but the gap is an artefact of scale, not of quality. iALS optimises a confidence-weighted reconstruction of a binary preference signal (Hu et al., 2008), while BPR optimises a pairwise ranking objective that is invariant to any monotonic rescaling of its scores (Rendle et al., 2009); neither produces an output that lives on [0.5, 5.0], so an RMSE of 2.3 or 3.5 measures the offset of an un-normalised score, not predictive error.

The deeper point is methodological. A low RMSE is a necessary diagnostic for the rating-oriented models, but it is a poor proxy for recommendation quality: Cremonesi et al. (2010) showed on MovieLens and Netflix that rating-error rankings and top-N rankings are essentially uncorrelated, and Steck (2013) traced the discrepancy to the interaction between error metrics and item popularity. Tables 2 and 3 confirm this on our own data.

4.2. Top-N ranking — full-catalogue Leave-One-Out

Model	HR@5	HR@10	HR@20	NDCG@5	NDCG@10	NDCG@20
Content-Based (Ridge CV)	0.015	0.024	0.041	0.010	0.012	0.017
User-Based (Pearson Baseline)	0.055	0.086	0.129	0.038	0.048	0.059
User-Based (Tuned, Jaccard)	0.050	0.075	0.113	0.032	0.040	0.050
iALS	0.016	0.028	0.051	0.010	0.013	0.019
BPR	**0.096**	**0.137**	**0.192**	**0.065**	**0.078**	**0.092**
BPR+Novelty	0.086	0.124	0.167	0.057	0.069	0.080

Table 2: Full-catalogue Leave-One-Out ranking. The higher the value, the better.

Against the full catalogue (Table 2), BPR is the clear leader (HR@10 = 0.137, NDCG@10 = 0.078), with its novelty variant a controlled step behind. The hierarchy is expected: BPR is the only model whose training signal is the ranking that HR and NDCG measure, so its pairwise objective aligns exactly with the evaluation (Rendle et al., 2009). What deserves comment is the bottom of the table. Content-Based, the RMSE champion, ranks last on retrieval, and iALS is barely above it. The inversion is the accuracy–ranking gap made concrete: a model can predict a held-out rating accurately yet fail to surface that item near the top of a list of nearly nine thousand candidates, because the absolute score and the relative position are different problems (Cremonesi et al., 2010).

The neighbourhood models occupy a sensible middle ground, recovering the target far more often than the content model precisely because they exploit co-rating structure rather than item attributes. Furthermore, it is worth noting that the reranking strategy applied to user-based models significantly enhanced their Leave-One-Out metrics, allowing them to achieve the third position overall. The ordering between the two neighbourhood variants is preserved here (HR@10 of 0.086 versus 0.075) and is proportionally wider than on rating error: the similarity metric sets the ceiling that the shared re-ranking step then lifts, so the Pearson Baseline's advantage on raw relevance carries through to the re-ranked list.

4.3. Top-N ranking — sampled protocol (1 positive + 99 negatives)

Model	HR@5 (*ns*)	HR@10 (*ns*)	HR@20 (*ns*)	NDCG@5 (*ns*)	NDCG@10 (*ns*)	NDCG@20 (*ns*)
Content-Based (Ridge CV)	0.226	0.315	0.462	0.158	0.187	0.224
User-Based (Pearson)	0.115	0.222	0.430	0.062	0.097	0.149
User-Based (Jaccard)	0.096	0.200	0.380	0.058	0.091	0.136
iALS	**0.712**	**0.857**	**0.904**	0.485	0.532	0.544
BPR	0.661	0.782	0.866	**0.495**	**0.535**	**0.556**
BPR+Novelty	0.625	0.751	0.817	0.469	0.511	0.528

Table 3: Negative-sampling Leave-One-Out (He et al., 2017): the held-out item is ranked against 99 randomly drawn unrated items, a fixed pool of 100 candidates per user.

Table 3 shows that restricting the candidate pool to one hundred items reshuffles the ranking, and the reshuffle is the point. iALS rises from near-last under the full catalogue (HR@10 = 0.028) to first under sampling ($HR@10_{ns} = 0.857$), a thirty-fold movement that no change in the underlying model can explain. The cause is the evaluation itself: Krichene and Rendle (2020) proved that sampled metrics are not order-preserving with respect to their full-catalogue counterparts, and that the distortion is largest for models whose score distribution correlates with item popularity. iALS, which concentrates confidence mass on a compact set of items, easily separates a single positive from 99 random negatives yet drowns that positive among thousands of competitors in the full ranking. BPR retains a marginal NDCG edge here even where iALS wins on HR, indicating that BPR places the target slightly higher when it is recovered, while iALS recovers it more often. Reporting both protocols side by side is what exposes this behaviour instead of concealing it; a report that quoted only the sampled numbers would have crowned iALS the best ranker, a conclusion Table 2 flatly contradicts.

4.4. Catalogue diversity (full mode, top-40)

Model	**Coverage (%) ↑**	**ILD ↑**	**MIUF ↑**
Content-Based (Ridge CV)	35.90	0.643	**5.878**
User-Based (Pearson Baseline)	6.07	**0.728**	1.252
User-Based (Tuned, Jaccard)	5.10	0.727	1.188
iALS	42.19	0.691	3.302
BPR	47.54	0.655	3.275
BPR+Novelty	**51.17**	0.650	3.627

Table 4: Diversity (ILD) and novelty (MIUF) in full mode (models trained on 100% of interactions, top-40 lists). Coverage is the share of the 8,737-item catalogue appearing in at least one list.

Table 4 reports coverage, ILD and MIUF in full mode. The diversity picture both confirms and corrects the project's working assumptions. BPR+Novelty delivers the widest catalogue coverage (51.17%) and the strongest long-tail exposure among the ranking models (MIUF = 3.627), exactly the effect its logarithmic popularity penalty was designed to produce (Abdollahpouri et al., 2019). The correction concerns intra-list diversity: ILD is essentially unchanged between BPR (0.655) and its novelty variant (0.650), and is in fact marginally lower. The benefit of the penalty therefore materialises as *catalogue-level* novelty and coverage, not as *within-list* genre spread, a distinction Vargas and Castells (2011) formalise as the difference between novelty and diversity, and one worth stating explicitly rather than claiming a uniform diversity gain.

Both user-based models — Pearson Baseline and Jaccard — exhibit the opposite pitfall. While they achieve the highest ILD of the benchmark (≈ 0.73), they suffer from a severe collapse in coverage ($\approx 5 - 6\%$) and record the lowest novelty by a wide margin. This behavior is characteristic of a system that recommends a narrow band of popular titles to almost everyone (Herlocker et al., 2004; Canamares and Castells, 2018).

Between the two variants the diversity profile is effectively identical (ILD 0.728 versus 0.727, coverage 6.1% versus 5.1%), which localises the cause: the catalogue collapse is a property of the neighbourhood mechanism itself, not of the similarity metric chosen to drive it. This sharp decline in novelty stems from a deliberate algorithmic shift. Prior to reranking, novelty metrics reached substantial heights, ranging from 4.3 to 5.3 and indicating a tendency to recommend niche films. Consequently, an intentional trade-off was made: reranking successfully optimized Leave-One-Out metrics at the expense of global catalog metrics. This deliberate compromise reflects a design choice aimed at tailoring carousel presentation to the user's limited attention span (Pei et al., 2019).

Content-Based stands apart again: the highest MIUF of all six models (5.878) confirms its tendency to reach into the long tail, the same property that grants it catalogue cold-start resistance and, simultaneously, exposes it to the filter-bubble risk noted in Section 3.3.2.

4.5. Synthesis

Read together, the four tables describe a portfolio rather than a podium. Rating accuracy belongs to Content-Based and, behind it, the Pearson neighbourhood model. Full-catalogue ranking belongs to BPR, catalogue novelty and coverage belong to BPR+Novelty and Content-Based. It is important to note that no model is strong everywhere. This is the diversity–accuracy dilemma that Zhou et al. (2010) characterised on MovieLens and Netflix: the objectives trade off against one another, and a system that wants all of them must obtain them from different engines. The deployment in Section 3.4 does exactly that, assigning Content-Based to *"Based On What You Like"* for its accuracy and explainability, the neighbourhood model to *"Viewers Like You Also Watched"*, iALS to *"Top Picks For You"*, and BPR+Novelty to *"Discover Something New"* for its long-tail reach. The benchmark does not select a winner; it justifies why four models coexist.

A final observation ties the metrics back to what the interface actually shows: the relationship between a diversity score and its on-screen effect is not uniform across models, and the user-based family illustrates this most sharply. Before re-ranking, their high MIUF (4.3–5.3) produced carousels dominated by obscure titles, nominally novel, but offering no recognisable anchor and therefore little immediate trust. Re-ranking deliberately lowered that novelty (MIUF ≈ 1.25) and, in doing so, lifted the Leave-One-Out metrics (Table 2) by surfacing familiar short-head titles first. The resulting carousel reads as less adventurous on paper yet more usable in practice: a lower diversity figure is not automatically a worse outcome, but the visible cost of a trade-off chosen for the carousel's first impressions (Pei et al., 2019). The same metric value carries a different meaning depending on the engine behind it.

5. Discussion & Conclusion

5.1. Summary

This work delivered a complete recommendation pipeline, from the evaluator block and analytics module of the early codings to a deployed carousel interface. Six models across three families: a per-user content-based ridge regressor, two user-based neighbourhood variants, and three implicit-feedback latent models (iALS, BPR, BPR+Novelty). All of these models are trained on MovieLens and assessed under three deliberately complementary protocols: rating prediction on a 75/25 split, full-catalogue Leave-One-Out ranking, and a sampled 1-vs-99 protocol, alongside catalogue-level diversity and novelty measures. The comparison fed directly into the deployment, which routes each model to the carousel its profile best serves. The global analysis refuses to crown a single model, and the architecture turns that refusal into its central asset: each model fails in a different place, and the system is built around those failures rather than against them.

5.2. Strengths

The decisive choice was to assemble a set of models whose weaknesses do not overlap. Rating accuracy and explainability come from the content-based regressor; full-catalogue ranking from BPR; long-tail coverage from its novelty variant; and the latent geometry of iALS both tops the sampled protocol and feeds the farthest-first onboarding. Each carousel is backed by the model genuinely best at its task, not by one engine stretched across incompatible objectives. Two further strengths are easy to undervalue. Because each content-based profile is a per-user ridge regression, its weights expose what drove a recommendation, an explanation the latent-factor models cannot give. And running full-catalogue and sampled ranking in parallel was not redundancy: it exposed a reordering of the hierarchy that a single protocol would have concealed (Krichene and Rendle, 2020).

5.3. Limitations

The evidence that flatters the system also bounds it. The neighbourhood models concentrate on a thin band of popular titles, and the re-ranking that lifts their top-of-list relevance buys that gain by spending novelty (Cañamares and Castells, 2018). More fundamentally, every figure here is offline, and the disagreement between the two ranking protocols is a warning in itself: offline winners do not reliably win with real users (Rossetti et al., 2016). Two structural gaps remain. Collaborative models cannot reach a film no one has rated, and neighbourhood similarity scales quadratically with the user base (Linden et al., 2003), leaving hackathon-tuned procedures impractical at production scale. Finally, the latent-factor models that top several of our protocols (iALS, BPR) give no interpretable account of why an item was recommended, in direct contrast to the per-user ridge weights of the content model. A limitation that bites precisely because explainability is one of the system's stated goals.

References

[1] Abdollahpouri, H., Burke, R., and Mobasher, B. (2019). Managing popularity bias in recommender systems with personalized re-ranking. *Proceedings of the 32nd International FLAIRS Conference*.

[2] Bell, R. M., and Koren, Y. (2007). Lessons from the Netflix prize challenge. *ACM SIGKDD Explorations Newsletter*, 9(2), 75–79.

[3] Cañamares, R., and Castells, P. (2018). Should I follow the crowd? A probabilistic analysis of the effectiveness of popularity in recommender systems. *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 415–424.

[4] Castells, P., Hurley, N. J., and Vargas, S. (2015). Novelty and diversity in recommender systems. In Ricci, F., Rokach, L., and Shapira, B. (Eds.), *Recommender Systems Handbook* (2nd ed., pp. 881–918).

[5] Cremonesi, P., Koren, Y., and Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. *Proceedings of the 4th ACM Conference on Recommender Systems (RecSys)*, pp. 39–46.

[6] Desrosiers, C., and Karypis, G. (2011). A comprehensive survey of neighborhood-based recommendation methods. In *Recommender Systems Handbook*, Springer, pp. 107–144.

[7] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T. S. (2017). Neural collaborative filtering. *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pp. 173–182.

[8] Herlocker, J. L., Konstan, J. A., Terveen, L. G., and Riedl, J. T. (2004). Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems (TOIS)*, 22(1), 5–53.

[9] Hoerl, A. E., and Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

[10] Hu, Y., Koren, Y., and Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 263–272.

[11] Jannach, D., Zanker, M., Felfernig, A., and Friedrich, G. (2015). *Recommender Systems: An Introduction*. Cambridge University Press.

[12] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.

[13] Krichene, W., and Rendle, S. (2020). On sampled metrics for item recommendation. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pp. 1748–1757.

[14] Linden, G., Smith, B., and York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1), 76–80.

[15] Lops, P., de Gemmis, M., and Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In *Recommender Systems Handbook*, Springer, pp. 73–105.

[16] Pan, R., Zhou, Y., Cao, B., Liu, N. N., Lukose, R., Scholz, M., and Yang, Q. (2008). One-class collaborative filtering. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 502–511.

[17] Pazzani, M. J., and Billsus, D. (2007). Content-based recommendation systems. In *The Adaptive Web*, Springer, pp. 325–341.

[18] Pei, C., Zhang, Y., Zhang, Y., Sun, F., Lin, X., Sun, H., Wu, J., Jiang, P., Ge, J., & Ou, W. (2019). Personalized re-ranking for recommendation. *In Proceedings of the 13th ACM Conference on Recommender Systems (RecSys '19) (pp. 3-11). ACM*

[19] Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 452–461.

[20] Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., and Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW)*, pp. 175–186.

[21] Rossetti, M., Stella, F., and Zanker, M. (2016). Contrasting offline and online results when evaluating recommendation algorithms. *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, pp. 31–34.

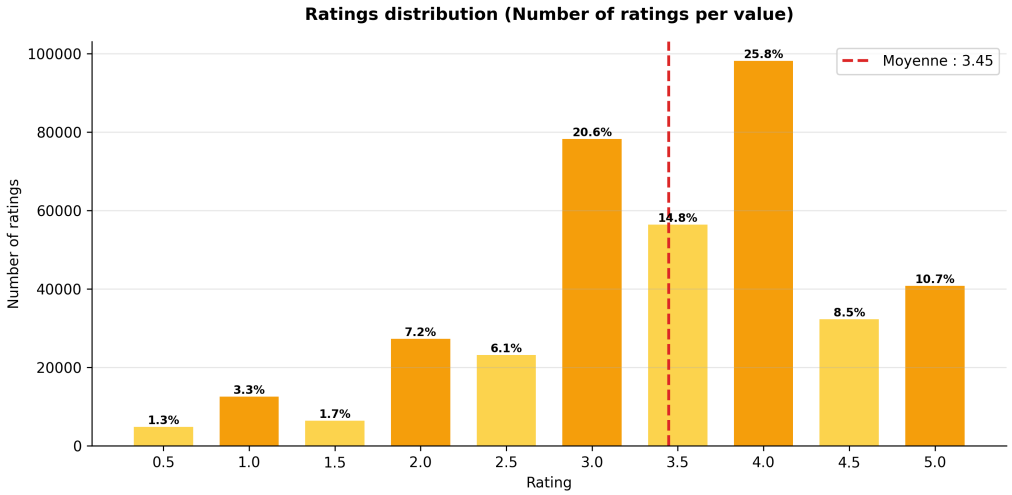
[22] Steck, H. (2013). Evaluation of recommendations: rating-prediction and ranking. *Proceedings of the 7th ACM Conference on Recommender Systems (RecSys)*, pp. 213–220.

[23] Vargas, S., and Castells, P. (2011). Rank and relevance in novelty and diversity metrics for recommender systems. *Proceedings of the 5th ACM Conference on Recommender Systems (RecSys)*, pp. 109–116.

[24] Zhou, T., Kuscsik, Z., Liu, J.-G., Medo, M., Wakeling, J. R., and Zhang, Y.-C. (2010). Solving the apparent diversity-accuracy dilemma of recommender systems. *Proceedings of the National Academy of Sciences (PNAS)*, 107(10), 4511–4515.

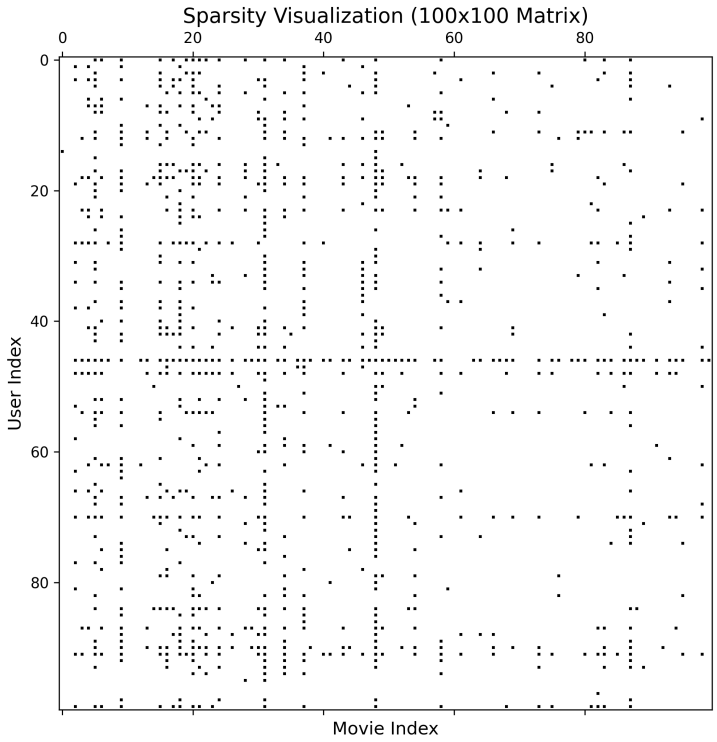
Appendix

Appendix 1 - Ratings distribution



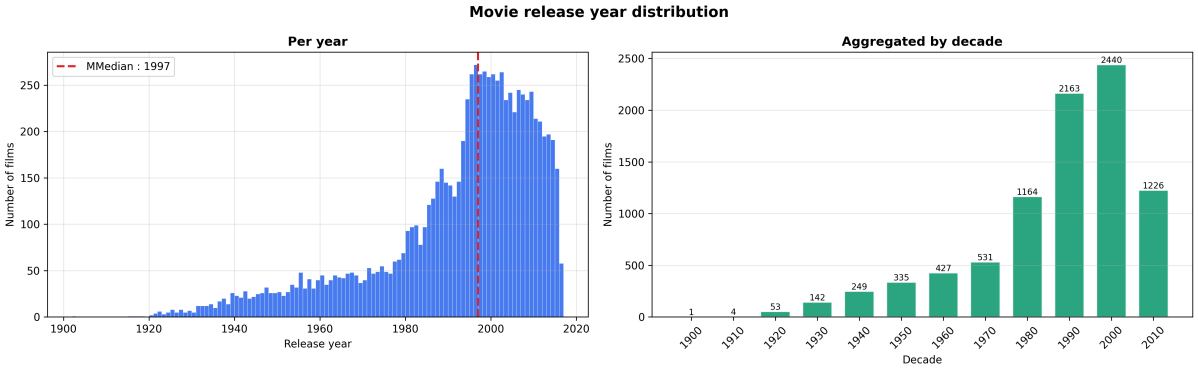
Appendix 1: Ratings distribution showing the total number of ratings per value, highlighting a global mean of 3.45 and a structural left-skewed positivity bias.

Appendix 2 - Sparsity Visualization



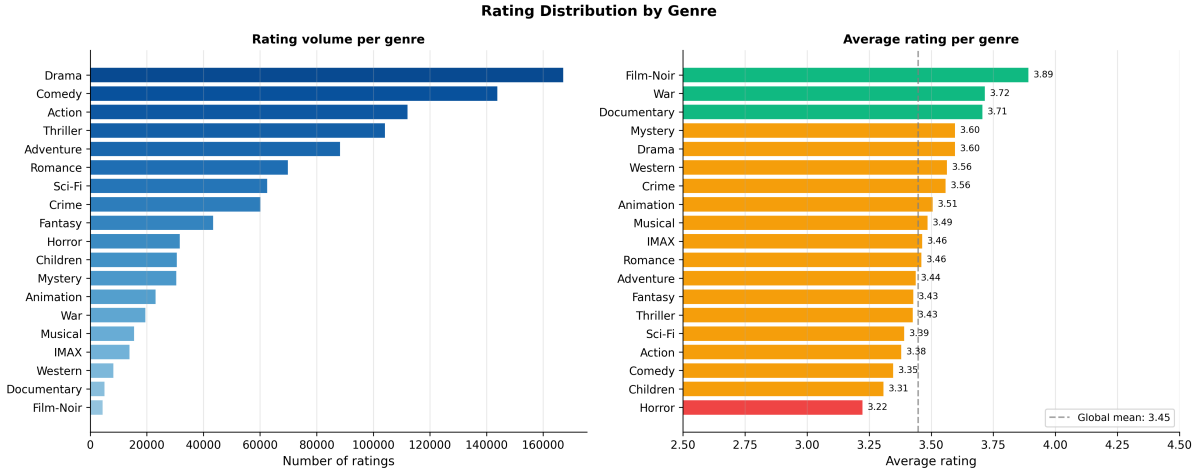
Appendix 2: Sparsity visualization mapping a 100x100 matrix slice, where black dots represent explicit user ratings and empty spaces highlight unobserved interactions.

Appendix 3 - Movie Release Year Distribution



Appendix 3: Movie release year distribution displaying a strong skew towards modern cinema, with a median release year of 1997 and a peak volume during the 2000s decade.

Appendix 4 - Rating Distribution by Genre



Appendix 4: Rating distribution by genre. The left panel displays the total number of ratings per genre. The right panel shows the average rating per genre.